KLAi

Classification: predicting a yes or no answer with logistic regression and a decision tree

How do we predict a category, and what does the model's confidence mean?

AI and Data Science Professional | Session 15 of 24 | Module D: Machine learning

KLAi Professional Learning | Jotham Martin Wambi

Today in one view: what you will leave with

By the end of today you can



Say what binary classification is, and fit a logistic regression and a decision tree on the Nile Microfinance loans



Read a predicted probability, and turn text variables into numbers with one-hot encoding



Choose a decision threshold from the cost of a missed default and the cost of a false alarm

Your two hours: concepts first, then a Python lab in Colab

0:00	10	Recall and the question of the day
0:10	45	Concepts: picture, plain words, one example by hand
0:55	10	Break
1:05	45	Lab: predict, run, read, change
1:50	10	Exit check, keep and next

Ideas with pictures first. Then you open the notebook. Then the checks and what you keep.

Three questions from last time

1. In the loan file, which variables are the features and which is the target?

2. Why do we hide a test set from the model?

3. What is a baseline model, and why do we compare against one?



Answer aloud first. Then we check together.

Answers to the three questions from Session 14

	Question	Short answer
	**In the loan file, which variables are the features and which is the target?**	The features are the inputs, such as income, loan amount and age. The target is default: 1 means the loan defaulted, 0 means it was repaid.
	**Why do we hide a test set from the model?**	To score the model on loans it has never seen, like a test with new questions. A score on the training set flatters the model.
	**What is a baseline model, and why do we compare against one?**	The simplest possible model. Here it says "no default" for every loan and is right 87.45 percent of the time (1,749 of the 2,000 test loans). Any model we build must beat it.

Words we use today

	Word	What it means
	**Binary classification (positive class, negative class)**	Predicting one of two categories. The positive class is the outcome we look for: default = 1. The negative class is the other: repaid = 0.
	**Logistic regression (sigmoid, log-odds, regularisation)**	A model that bends a score from the features into a probability between 0 and 1 with the sigmoid. scikit-learn penalises large coefficients (regularisation, C = 1.0).
	**Decision tree (root node, leaf node)**	A model made of yes or no questions. Each path ends in a leaf node, and the leaf gives the predicted probability.
	**Decision threshold (cut-off)**	The probability at or above which we predict the positive class. The default of 0.5 is rarely right for a business.
	**One-hot encoding**	Turning a text variable into one 0 or 1 indicator column per category, so a model can use it.

Binary classification predicts one of two categories for each loan

loan_id	occupation	loan_amount_ugx	instalment_to_income	default
N00001	Mechanic	1,583,200	0.60	0
N00003	Trader	5,081,300	4.73	1
N00002	Teacher	3,366,600	0.62	0
N00013	Farmer	811,200	0.27	1

features: the inputs, known when the loan is approved

target (label): the answer

default = 1: the loan defaulted (positive class)

default = 0: the loan was repaid (negative class)

Classification predicts a category. Binary classification has exactly two.

Positive class: the outcome we look for. Here default = 1.

Negative class: the other outcome. Here repaid, default = 0.

In Excel: a Yes or No column to fill in for new loans.

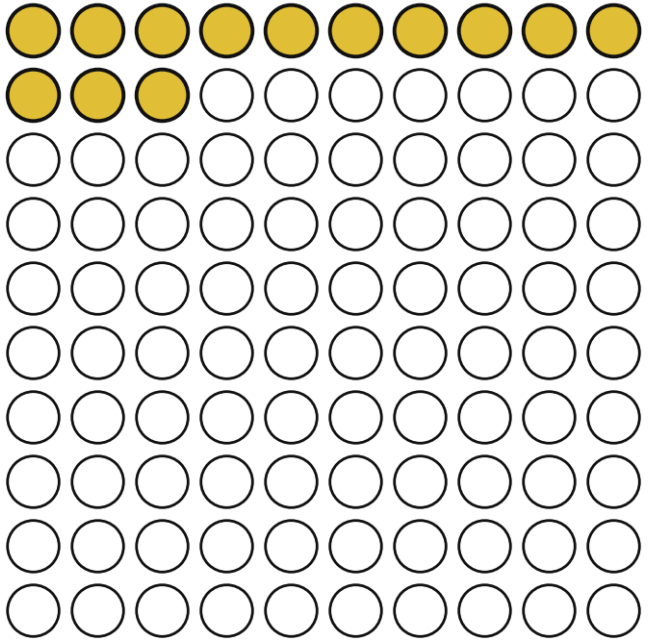
The spreadsheet word and the machine learning word for each idea

	In a spreadsheet	In machine learning	Plain meaning
	**A Yes or No column**	Target (label)	The answer we want to predict for each record.
	**A percentage beside each row**	Predicted probability	The model's chance, from 0 to 1, that the record is in the positive class.
	**=IF(score>=0.2,"Yes","No")**	Decision threshold	The probability at or above which we say yes.
	**A helper column per category**	One-hot encoding	One 0 or 1 indicator column for each category.
	**A trendline that returns a probability**	Logistic regression	A linear score bent into a probability.
	**A nested IF written by the computer**	Decision tree	Yes or no questions that end in a verdict.

Only 12.6 percent of loans default, so the positive class is rare

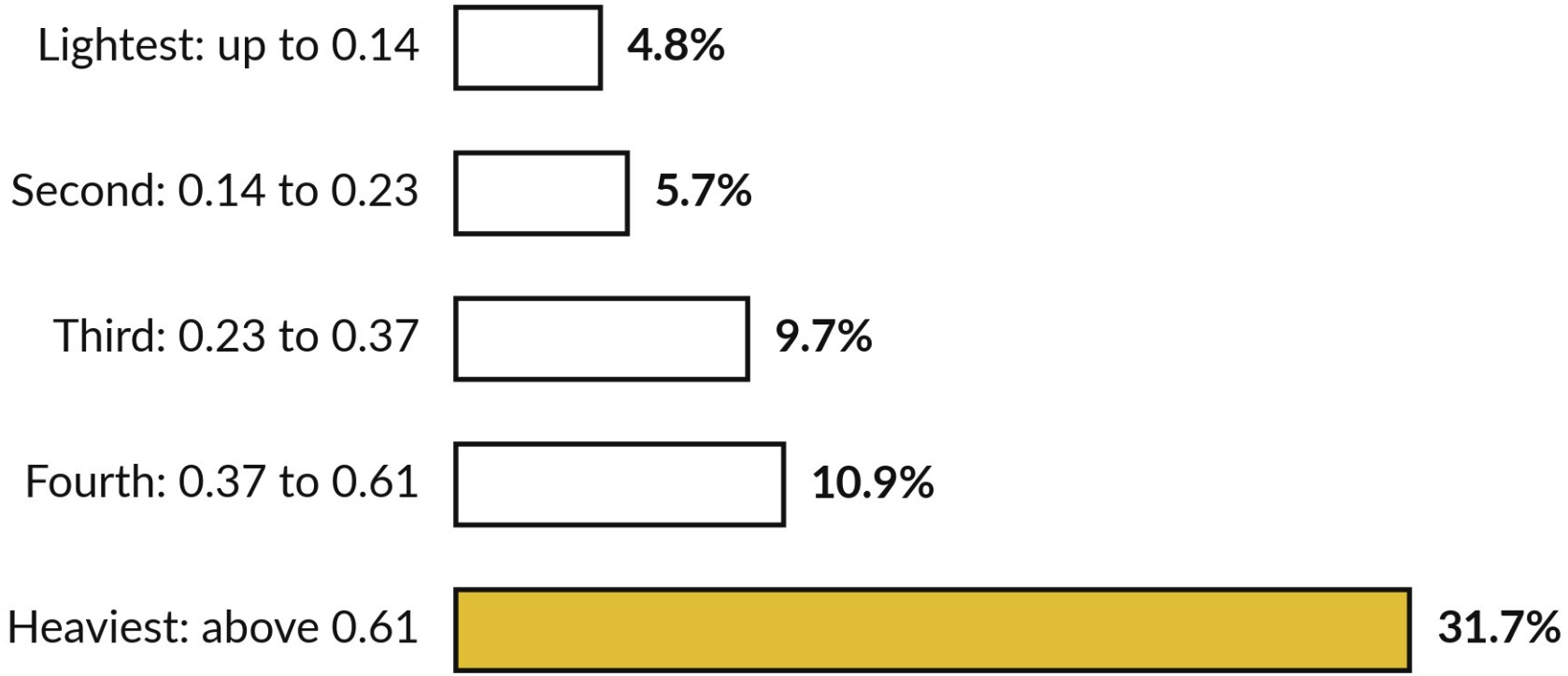
13%

1,004 of the 8,000 loans defaulted. About one loan in eight is in the positive class.



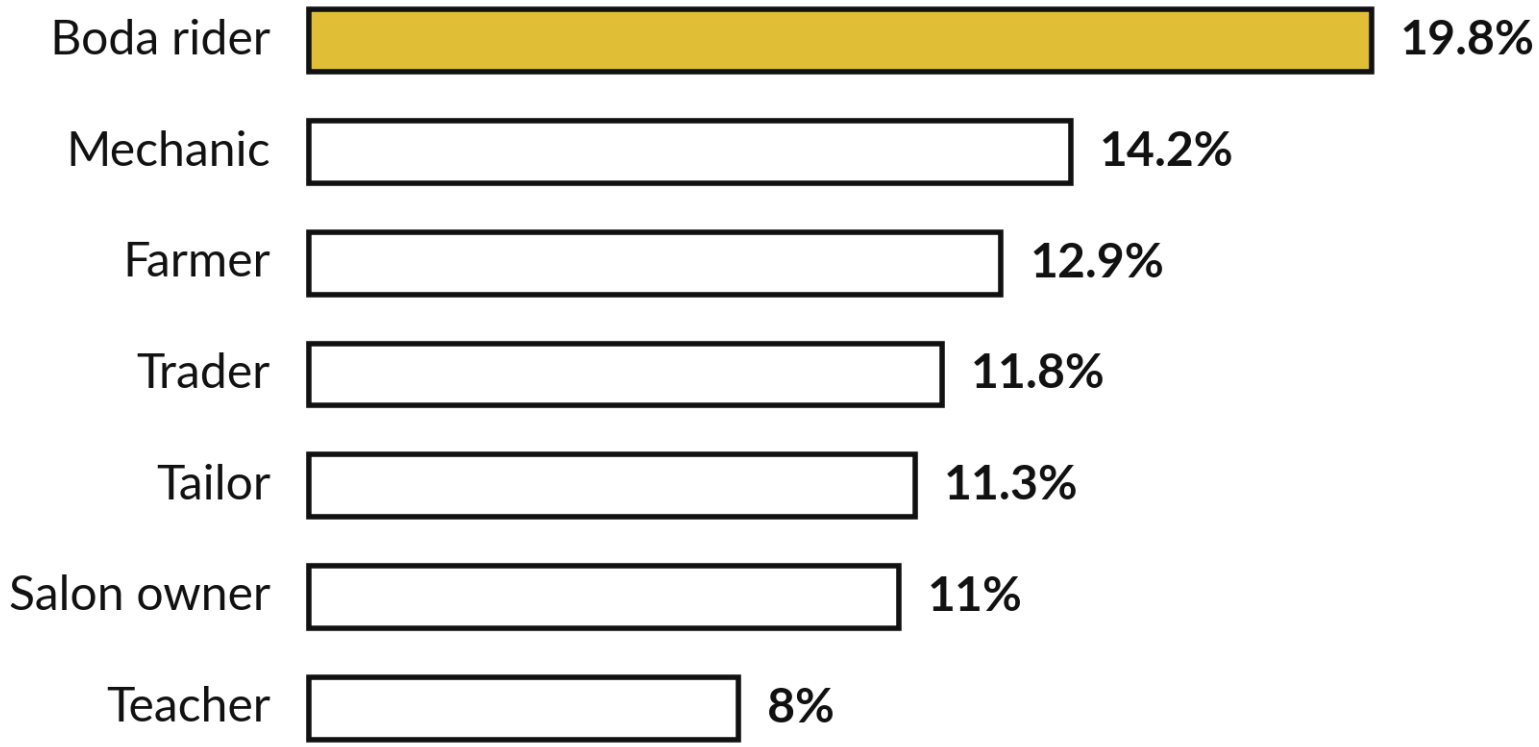
About 13 of every 100 loans defaulted

Default rate climbs with the instalment-to-income ratio



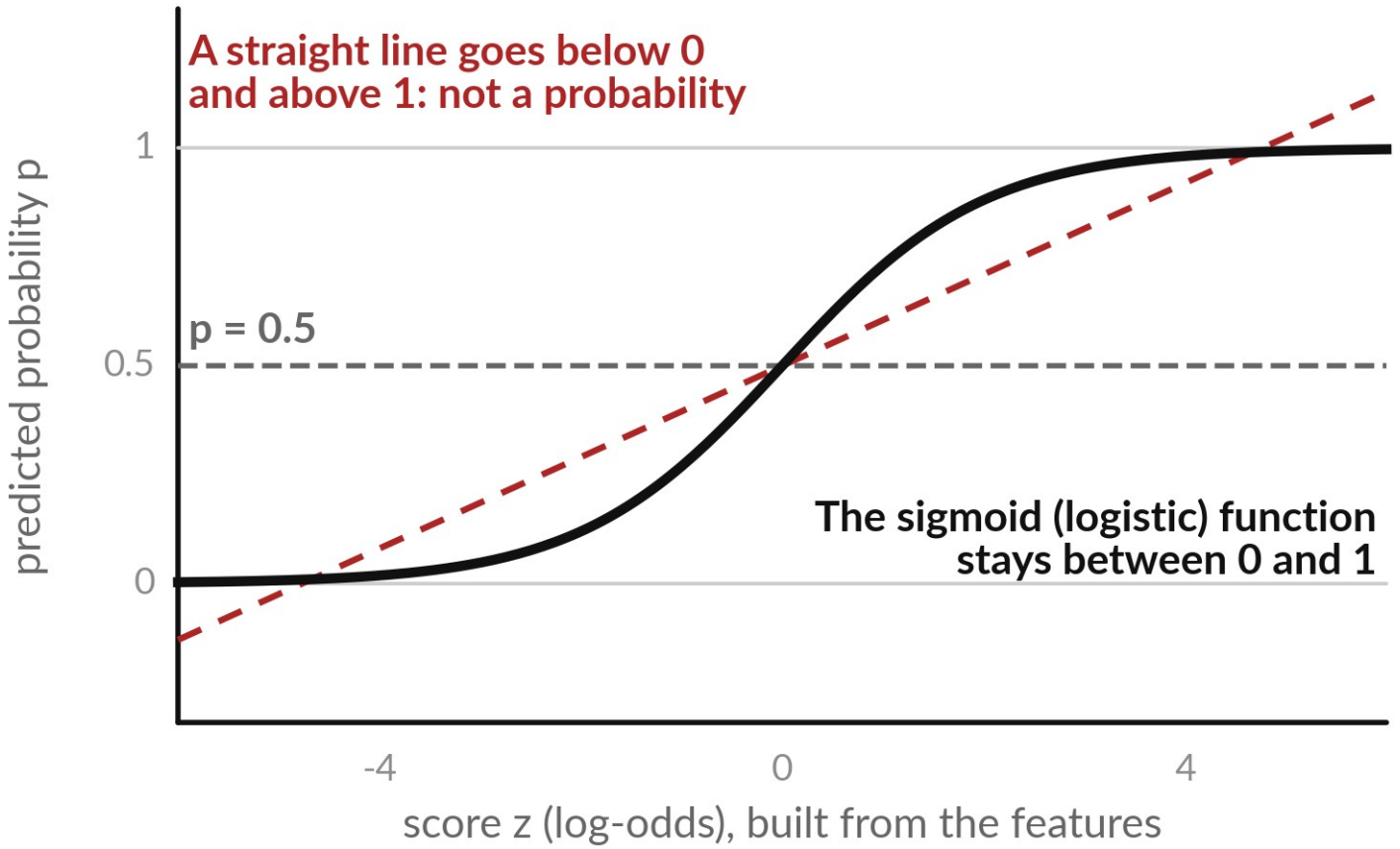
Five equal groups of 1,600 loans. The 1.15 in the ratio is assumed.

Occupation is text, yet it carries a signal a model can use



Share of loans that defaulted in each occupation. A model needs these words as numbers.

A straight line can leave the range 0 to 1, so logistic regression bends it with the sigmoid



A probability must stay between 0 and 1. A straight line does not.

The sigmoid (logistic) function squeezes any score into 0 to 1.

Logistic regression: a linear score, then the sigmoid.

Despite the word regression, it is a classification model.

Logistic regression: a linear score z becomes a probability p by the sigmoid

score

$$z = b_0 + b_1 x_1 + b_2 x_2 + \cdots + b_p x_p$$

probability

$$p = \frac{1}{1 + e^{-z}}$$

Symbol	What it means	In our data
z	The score, also the log-odds ln(p / (1 - p))	-1.12 for loan N04300
p	Predicted probability of default. In Excel: =1/(1+EXP(-z))	0.25 for that loan
b_0	Intercept: the score when every feature is 0	-2.23
b_1 ... b_p	One coefficient per feature (per 1 standard deviation), learned from the training set	ratio: +0.80
x_1 ... x_p	The feature values of one loan, standardised (z-scores)	ratio: +0.82

Log-odds is another way to write a probability: 0 means even odds

	Probability p	Odds = p / (1 - p)	In words	Log-odds z
🟡	**0.05**	0.053	1 to 19 against	-2.94
🟡	**0.10**	0.111	1 to 9 against	-2.20
🟡	**0.20**	0.250	1 to 4 against	-1.39
✅	**0.50**	1.000	even: 1 to 1	+0.00
📈	**0.80**	4.000	4 to 1 in favour	+1.39

Odds are what bookmakers quote. A positive log-odds means the positive class is more likely than not.

One loan, step by step: from features to a predicted probability

Loan N04300 from the test set: Boda rider, instalment-to-income ratio 0.79, late payments 0.

1 Score z	`Intercept -2.23, ratio +0.65, late payments -0.17, Boda rider +0.41, 24 other terms +0.22. Sum z = -1.12.`
2 Probability	`p = 1 / (1 + e^1.12) = 0.25: a 25 percent chance of default.`
3 Decision	At threshold 0.20 the loan is flagged. At 0.50 it is not.
4 Reality	In the data this loan defaulted. One loan proves nothing: we judge all 2,000 test loans.

A coefficient shows the direction of an effect, and its exponent shows the change in odds per standard deviation

	Variable	Coefficient b	Odds ratio e^b	Reading
	**instalment_to_income**	+0.80	2.23	Per standard deviation (about 0.44): odds multiplied by this much.
	**previous_late_payments**	+0.26	1.30	Each late-payment step raises the odds.
	**occupation_Boda rider**	+0.13	1.14	Higher odds than the other occupations.
	**occupation_Teacher**	-0.02	0.98	Lower odds than the other occupations.
	**savings_balance_ugx**	-0.18	0.83	More savings: lower odds.
	**group_loan**	-0.16	0.85	Group loans have lower odds.

Per 1 standard deviation, other features fixed. scikit-learn adds a penalty by default (regularisation, C = 1.0), so a different scaling or C changes them.

One-hot encoding turns each occupation into its own column of 0 and 1

Before: text

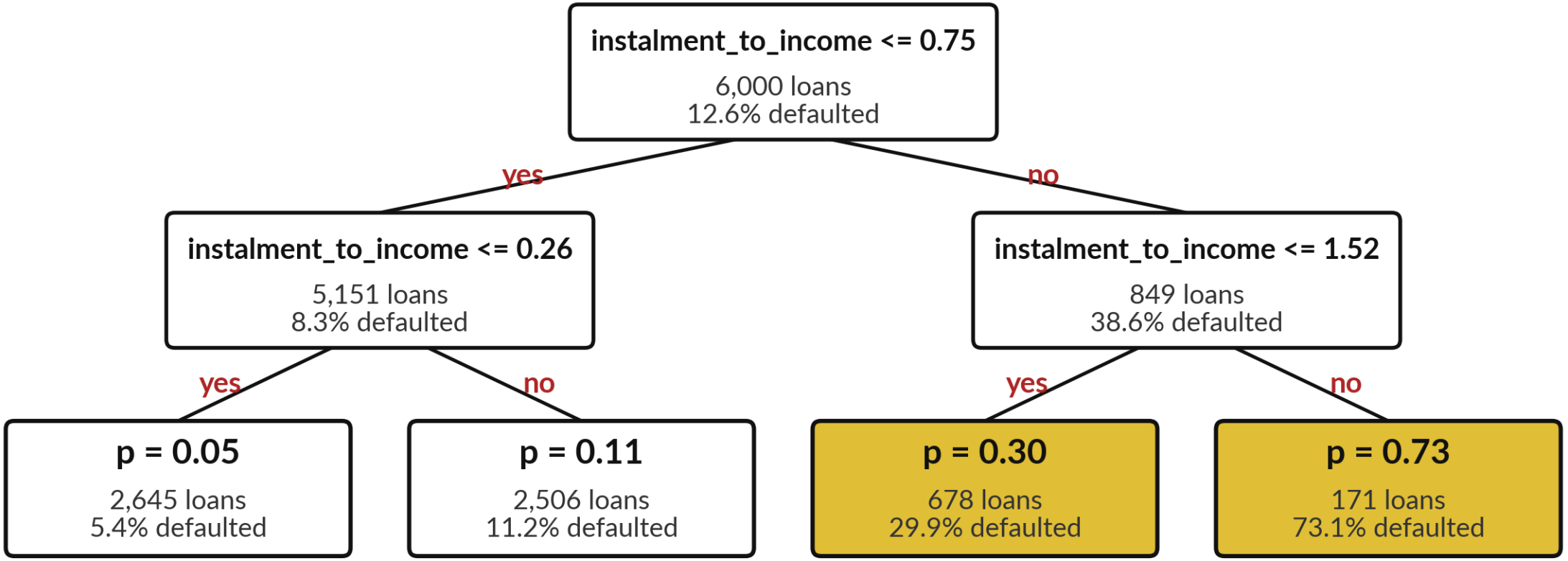
occupation
Mechanic
Teacher
Trader

After: one 0 or 1 column per category

Boda rider	Mechanic	Teacher	Trader	+3 more
0	1	0	0	0 …
0	0	1	0	0 …
0	0	0	1	0 …

get_dummies makes 15 indicator columns from 3 text variables: 12 columns become 27.

A decision tree (depth 2) asks yes or no questions, and each leaf gives a predicted probability



Depth 2 (the lab tree has one more level), 6,000 training loans. Gold leaf: 25 percent or more default.

A tree has a root node, internal nodes and leaf nodes joined by splits

Term	Plain meaning	In our tree
Root node	The first question, asked of every training loan.	instalment_to_income <= 0.75, asked of 6,000 loans
Split	One question on one feature with a threshold. The yes and no answers send loans down two branches.	instalment_to_income <= 0.75
Internal node	A question asked after the first one.	instalment_to_income <= 0.26
Leaf node	An end point with no further question. Its predicted probability is the share of its training loans that defaulted.	p from 0.05 to 0.90
Depth	The most questions on any path from root to leaf. We set max_depth to stop the tree growing without limit.	3 questions, so 8 leaves

A tree chooses each split to make the two groups purer, measured by Gini impurity

Gini impurity

$$G = 1 - \sum_k p_k^2$$

after a split

$$G_{split} = \frac{n_L}{n} G_L + \frac{n_R}{n} G_R$$

Symbol	What it means	In our data
G	Impurity: 0 is pure, 0.5 is an even mix	Root: 0.219
p_k	Share of the group in class k	12.6% default at the root
n_L, n_R	Loans sent left and right by the split	5,151 and 849
G_L, G_R	Impurity of each group	0.151 and 0.474
G_split	Weighted average of the two groups: the lowest wins	0.197, down from 0.219

A nested IF and a decision tree ask the same kind of question

A nested IF you write

=IF(ratio>1.5,"flag","clear")

You choose the thresholds from experience.

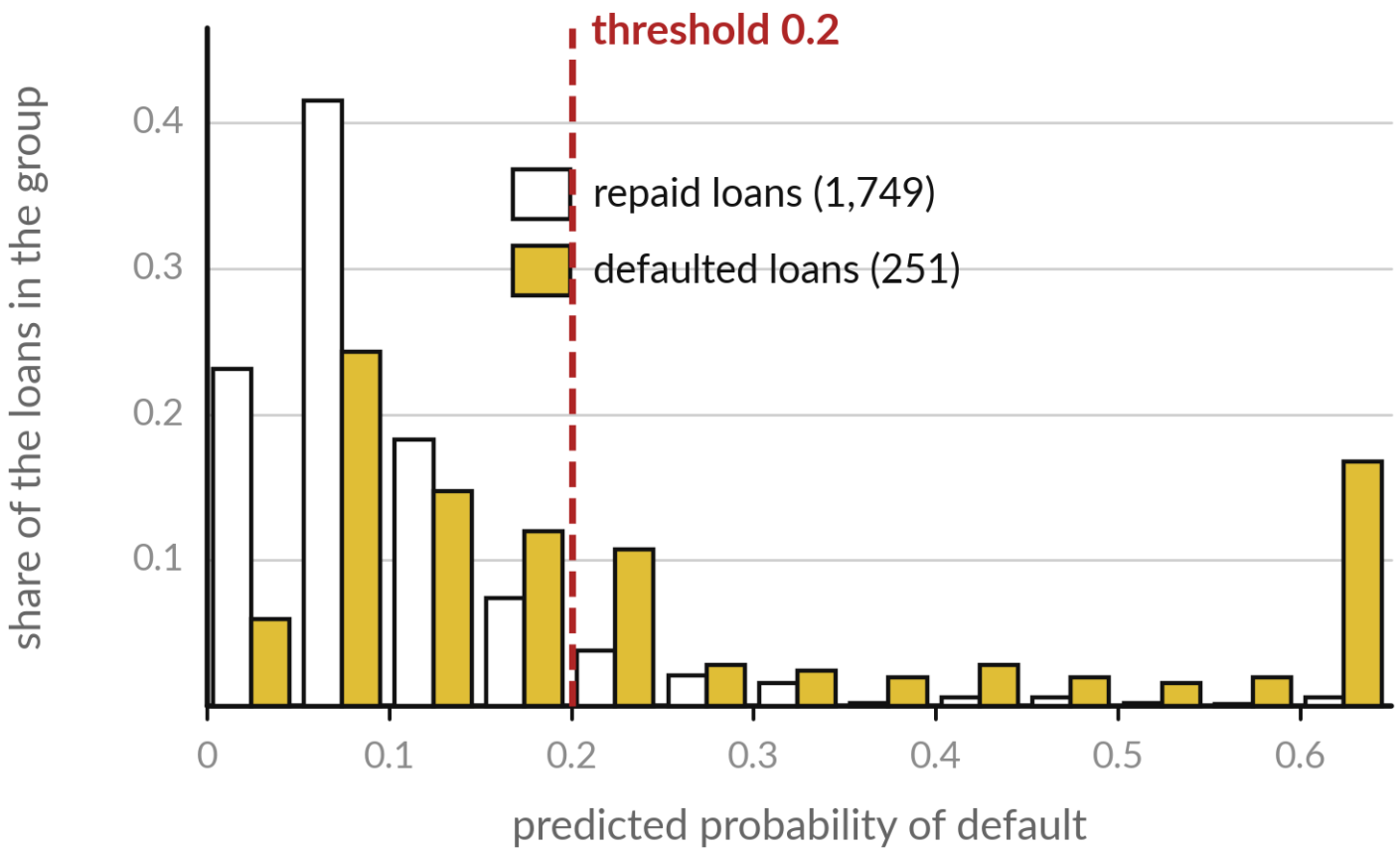
A tree the computer writes

It finds the thresholds that best separate defaulted from repaid loans.

Our tree starts with instalment_to_income <= 0.75.

Same shape. The tree finds the thresholds from examples.

The model gives a predicted probability, and the decision threshold turns it into yes or no



Flag a loan when its probability is at or above the threshold.

Defaulted loans sit further right, but the groups overlap.

At 0.20, 43 percent of defaulted loans are flagged, and 10 percent of repaid loans too.

scikit-learn predict() uses 0.5. You can choose another.

Moving the decision threshold trades missed defaults for false alarms

	Threshold	Loans flagged	Defaults caught (TP)	False alarms (FP)	Defaults missed (FN)
🔒	**0.50**	67	51	16	200
🔒	**0.30**	141	74	67	177
💊	**0.20**	278	108	170	143
🔔	**0.15**	437	138	299	113
🔔	**0.10**	794	175	619	76

Logistic regression on the 2,000 test loans, of which 251 defaulted. For reading only: we do not pick a threshold from this table. TP, FP, FN: true positive, false positive, false negative.

The two mistakes cost different amounts, so the threshold is a business decision

A missed default

A false negative: the loan is not repaid.

Illustrative cost: UGX 1,000,000, the principal lost.

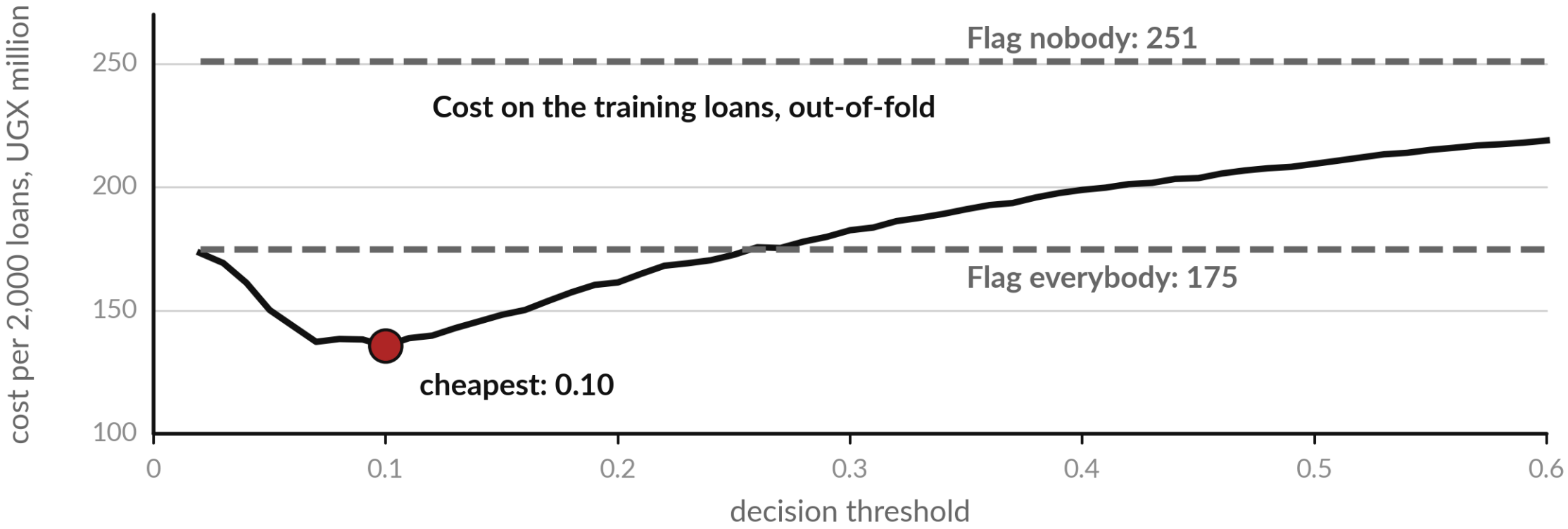
A false alarm

A false positive: a good borrower is refused.

Illustrative cost: UGX 100,000, the interest not earned.

These two costs are teaching numbers. Use your own lender's.

Cross-validation on the training loans finds a threshold of 0.10, close to the cost rule 0.0909



Training loans, 5-fold. Illustrative: UGX 1,000,000 per missed default, UGX 100,000 per false alarm.

The test loans are used once, to report the cost of the choices already made

	Choice	Threshold	Flagged	Caught	False alarms	Missed	Cost, UGX million
	**Default of predict()**	0.50	67	51	16	200	201.6
	**Cross-validation, training loans**	0.10	794	175	619	76	137.9
	**Cost rule, no data needed**	0.0909	897	188	709	63	133.9
	**Flag nobody**	-	0	0	0	251	251.0
	**Flag everybody**	0	2,000	251	1,749	0	174.9

Illustrative costs, 2,000 test loans, 251 defaults. Searching the test loans would find 0.08 at UGX 133.1 million, a gain of under UGX 5 million that exists only on these loans. We do not do that.

Where you will use this at work: score every record, then set the decision threshold from the cost

	Your job	The yes or no question	What today lets you do
	**Credit risk analyst at a microfinance institution, using scikit-learn**	Will this loan default?	Score every application with a probability and show the manager the cost of each decision threshold in UGX.
	**Fraud analyst at a mobile money provider, using Python and SQL**	Is this transaction suspicious?	Choose how many cases go to a human reviewer by moving the threshold.
	**Programme officer at a health organisation, using Excel and Python**	Will this patient miss the follow-up visit?	Rank patients by probability so nurses phone the highest first.
	**Sales analyst handing results to Power BI**	Will this customer stop buying?	Export a probability column and a flag column that a dashboard can filter.

Read the recipe before the lab: six lines fit and use both models

```
X = pd.get_dummies(features, dtype=int)
model = make_pipeline(StandardScaler(),
    LogisticRegression(max_iter=1000))
model.fit(X_train, y_train)
y_prob = model.predict_proba(X_test)[:, 1]
tree = DecisionTreeClassifier(
    max_depth=3, random_state=42)
```

In plain words

Line 1: one-hot encode the text. 12 numeric columns become 27.

Lines 2 and 3: a StandardScaler, then logistic regression with its defaults (L2 penalty, C = 1.0).

Line 4: fit on the training loans.

Line 5: column 1 of predict_proba is the probability of default.

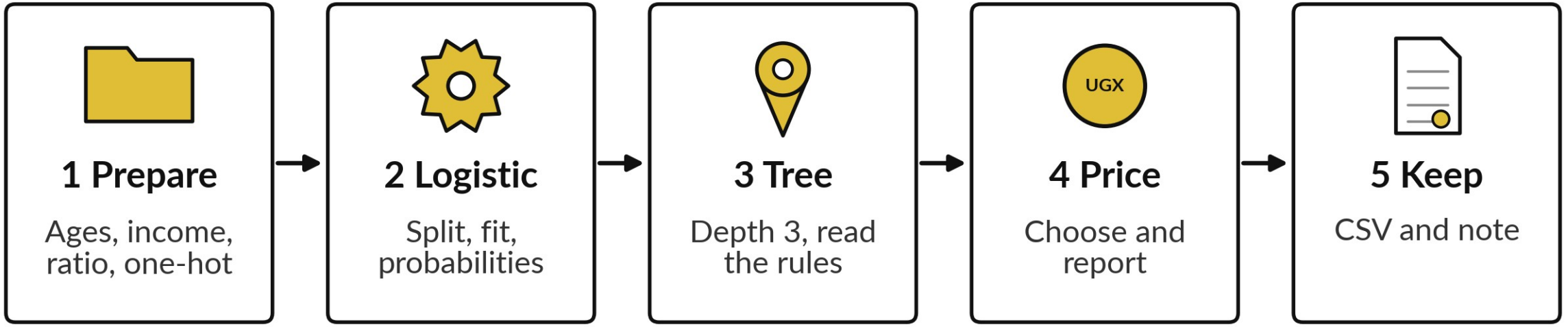
Lines 6 and 7: a tree with at most three questions.



Part 2: the Python lab

Close the slides. Open the notebook.

The lab has five steps, and you keep a scored file and a threshold note



Predict, run, read, change. Step 5 writes loan_risk_scores.csv and your threshold note.

Check your numbers before you move on

What you compute	You should get
Loans, and the default rate	8,000, and 12.6 percent
Columns of X after one-hot encoding	27
Loans in X_train and X_test, and defaults in y_test	6,000, 2,000 and 251
Coefficient of the ratio in the logistic regression	about +0.80
Tree depth, leaves, and the root question	3, 8 and instalment_to_income <= 0.75
Logistic regression at 0.20: flagged, caught, false alarms	about 278, 108 and 170
Threshold from 5-fold cross-validation, and the cost rule	about 0.10, and 0.0909
Cost on the test loans at the cross-validated threshold	near UGX 138 million

scikit-learn may differ from these by a few loans or a unit in the second decimal. If a number is far away, find the step that went wrong.

Five likely mistakes in the lab and how to fix them

What you see	What it means	What to do
ValueError: could not convert string to float	A text variable such as occupation is still in X.	Run pd.get_dummies on the features before fitting.
KeyError on a column name	A misspelt or already dropped column.	Print df.columns and copy the exact name.
A probability above 0.9 for many loans	You used the leaking column collection_calls.	Drop it. Session 16 explains why.
predict_proba gives two columns and the numbers look backwards	Column 0 is the probability of repaid.	Take column 1 for the probability of default.
ValueError: Found input variables with inconsistent numbers of samples	X and y have different row counts, for example one was filtered and the other not.	Filter both, or filter before you split.

Exit check: three questions, then compare with the short answers

Question	Short answer
A model gives a loan p = 0.30. What does that number mean, and what does a threshold of 0.20 do with it?	A 30 percent chance of default, as the model estimates. At threshold 0.20 the loan is flagged, because 0.30 is at or above 0.20.
Why do we one-hot encode occupation before fitting?	The model works on numbers. One-hot encoding gives each occupation its own 0 or 1 column, with no false order between them.
A missed default costs ten times a false alarm. Should the threshold be higher or lower than 0.50, and why?	Lower. Missing a default is the dearer mistake, so flag more loans and accept more false alarms.

Save your notebook and your threshold note

What you keep from today

Your notebook, a file loan_risk_scores.csv with a probability and a flag for each of the 2,000 test loans, and a three-line threshold note with its cost in UGX.

Our question today

How do we predict a category, and what does the model's confidence mean?

Next session

Session 16: Judging a model. We measure precision, recall and AUC, catch a column that leaks the answer, and test with cross-validation.

Take away

Your lab files

Notebook and CSV

The decision

A priced threshold

Next session

Judging a model